EE1030 : SOFTWARE ASSIGNMENT

EE25BTECH11018 – Darisy Sreetej

IMAGE COMPRESSION USING TRUNCATED SVD

I. Introduction

Image compression aims to minimize storage requirements by expressing image data with fewer elements while maintaining visual quality. Through Singular Value Decomposition (SVD), an image matrix can be reconstructed using only its dominant singular values. This approach effectively preserves the main structure and details of the image in a compact form. The project applies truncated SVD to compress grayscale images and evaluates its accuracy for various rank values k

II. Summary of Strang's video

The Singular Value Decomposition (SVD) is a fundamental way to break down any real matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ into simpler components as

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^{\mathbf{T}}$$

where $\mathbf{U}$, $\mathbf{V}$ are orthogonal matrices satisfying $\mathbf{U}^{\mathbf{T}}\mathbf{U} = \mathbf{I}_{\mathbf{m}}$ and $\mathbf{V}^{\mathbf{T}}\mathbf{V} = \mathbf{I}_{\mathbf{n}}$ The columns of $\mathbf{U}$ are called the left singular vectors, and they represent orthogonal directions in the output space, while the columns of $\mathbf{V}$ are the right singular vectors, representing orthogonal directions in the input space. The diagonal matrix $\mathbf{S}$ contains singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$,which indicate the relative importance of each pair of singular vectors.

III. Algorithms

Various algorithms exist to compute eigen values and eigen vectors. The following shows some widely used algorithms

1) **QR Algorithm :** This algorithm iteratively decomposes $\mathbf{A} = \mathbf{QR}$ (where $\mathbf{Q}$ is orthogonal and $\mathbf{R}$ is upper triangular) and updates $\mathbf{A} \leftarrow \mathbf{R} \cdot \mathbf{Q}$. The process converges to a triangular matrix , with eigen values on its diagnol.
2) **Power iteration :** This iterative algorithm computes the dominant eigenvalue (largest by magnitude) and its corresponding eigenvector. Starting with an arbitrary vector $\mathbf{x}$, the method repeatedly multiplies $\mathbf{A} \cdot \mathbf{x}$ and normalizes the result.
3) **Jacobi method :** Designed for symmetric matrices, this algorithm zeroes out off-diagonal elements iteratively using Givens rotations. It converges to a diagonal matrix with eigenvalues on the diagonal.
4) **Schur Decomposition :** The Schur decomposition decomposes a matrix $\mathbf{A}$ as $\mathbf{A} = \mathbf{QTQ}^*$, where $\mathbf{Q}$ is a unitary matrix and $\mathbf{T}$ is a upper triangular matrix . The eigen values of $\mathbf{A}$ are found on diagnols of $\mathbf{T}$

IV. Why do I choose Power iteration ?

I selected Power Iteration method because because it offers a simple and intuitive way to estimate the largest singular values and their corresponding vectors without involving complicated matrix factorizations.It is computationally effecient for large , sparse matrices. The method relies on basic operations such as matrix vector multiplication and normalization, making it easy to implement and understand. Overall, it provides a good balance between accuracy and computational cost, which makes it an ideal choice for truncated SVD based image compression.

V. Power iteration

V.1 MATH

To find the Truncated SVD (Singular Value Decomposition) of an image matrix $\mathbf{A}$, which is represented by the approximation

$$\mathbf{A} = \mathbf{USV^T}$$

The singular values of $\mathbf{A}$ (σ_i) are the square roots of the eigenvalues (λ_i) of the matrix $\mathbf{B} = \mathbf{A^T A}$.The right singular vectors of $\mathbf{A}$ (v_i) are the eigen vectors of matrix $\mathbf{B}$.Since $\mathbf{B}$ is a symmetric positive semi-definite matrix, the Power Iteration method can be efficiently used to find its dominant (largest) eigenvalue and eigenvector.
The standard Power Iteration algorithm iteratively approximates the dominant eigenvalue λ_1 and its corresponding eigenvector $\mathbf{v_1}$ of a symmetric matrix $\mathbf{B} \in \mathbb{R}^{n \times n}$.

1) **Initialization:** Choose an arbitrary, non-zero starting vector $\mathbf{v^{(0)}}$.
2) **Iteration:** The core iterative step is defined by:

$$\mathbf{w^{(j+1)}} = \mathbf{B v^{(j)}}$$

$$\mathbf{v^{(j+1)}} = \frac{\mathbf{w^{(j+1)}}}{\left\|\mathbf{w^{(j+1)}}\right\|}$$

   where j is the iteration count, $\mathbf{v^{(j)}}$ is the current eigenvector estimate, and $\left\|\mathbf{w^{(j+1)}}\right\|$ is the Euclidean (or L_2) norm used for normalization. The normalization step ensures numerical stability and prevents the vectors from growing indefinitely.
3) **Eigenvalue Estimation:** As $j \to \infty$, $\mathbf{v^{(j)}}$ converges to the dominant eigenvector $\mathbf{v_1}$. The dominant eigenvalue λ_1 can be estimated using the Rayleigh quotient:

$$\lambda_1 \approx \frac{(\mathbf{v^{(j)}})^T \mathbf{B v^{(j)}}}{\left\|\mathbf{v^{(j)}}\right\|} = (\mathbf{v^{(j)}})^T \mathbf{B v^{(j)}}$$

   (Since $\mathbf{v^{(j)}}$ is a unit vector, $\left\|\mathbf{v^{(j)}}\right\|$.
4) **Deflation :** To find the subsequent eigenpairs $(\lambda_2, \mathbf{v_2})$, $(\lambda_3, \mathbf{v_3})$, and so on, we use **Deflation** technique. Once the t-th eigenpair $(\lambda_t, \mathbf{v_t})$ is found, we subtract its contribution from the original matrix $\mathbf{B}$ to create a new matrix $\mathbf{B}^*$ that has the same eigenvectors as $\mathbf{B}$, but where λ_t is replaced by zero.
   The deflated matrix $\mathbf{B_{t+1}}$ is computed from the previously used matrix $\mathbf{B_t}$ and the calculated eigenpair $(\lambda_t, \mathbf{v_t})$ as follows:

$$\mathbf{B_{t+1}} = \mathbf{B_t} - \lambda_t \mathbf{v_t} \mathbf{v_t^T}$$

   where $\mathbf{B_1} = \mathbf{B}$. The Power Iteration is then applied to the deflated matrix $\mathbf{B_{t+1}}$ to find the next largest eigenpair, $(\lambda_{t+1}, \mathbf{v_{t+1}})$.

5) **SVD Calculation :** After computing the t-th eigenpair $(\lambda_t, \mathbf{v_t})$ of $\mathbf{B} = \mathbf{A^T A}$:
The t-th singular value is $\sigma_t = \sqrt{\lambda_t}$. The t-th right singular vector is $\mathbf{v_t}$ (the normalized eigenvector).
The t-th left singular vector $\mathbf{u_t}$ is computed using the SVD definition $\mathbf{A v_t} = \sigma_t \mathbf{u_t}$:

$$\mathbf{u_t} = \frac{\mathbf{A v_t}}{\sigma_t}$$

This vector $\mathbf{u_t}$ must also be normalized to ensure it is a unit vector.

V.2 PSEUDO-CODE

1) **Function 1: (normalization)**
Input: vector v of length n

Output: normalized vector v (unit length)
Procedure:
1. Compute magnitude = sqrt(sum of $\mathbf{v[i]^2}$ for all i)
2. If magnitude > EPS:
For each element i : $\mathbf{v[i]}$ = $\mathbf{v[i]}$ / magnitude

2) **Function 2: (matrix-vec)**
Input: matrix $\mathbf{M}$ of size $m * n$, vector $\mathbf{v}$ of length n

Output: result = $\mathbf{M} \cdot \mathbf{v}$(length m)
Procedure:
For each row i from 0 to $m - 1$:
$\mathbf{result[i]} = 0$
For each column j from 0 to $n - 1$:
$\mathbf{result[i]} += \mathbf{M[i * n + j]} * \mathbf{v[j]}$

3) **Function 3 : (pow-iter)**
Input: square matrix $\mathbf{N}$ $(n * n)$

Output: dominant eigenvalue λ and eigenvector $\mathbf{v}$

Procedure:
1. Initialize $\mathbf{v}$ randomly and normalize($\mathbf{v}$)
2. Repeat for MAX iterations:
$\mathbf{x} = \mathbf{N} \cdot \mathbf{v}$
normalize($\mathbf{x}$)
$\mathbf{v} = \mathbf{x}$
3. Compute $\mathbf{x} = \mathbf{N} \cdot \mathbf{v}$
4. eigen-val = $\mathbf{v^T} \cdot \mathbf{x}$ (Rayleigh quotient)
5. Return eigen-val (largest eigenvalue)

4) **Funtion 4 : (svd-reconstructed)**
Input:
$\mathbf{A}$: matrix of size $(\mathbf{r} \cdot \mathbf{c})$
k : number of singular components to retain

Output:
A-recon : rank-k reconstructed approximation of A
Procedure:

1. Allocate matrices:
B = $c * c$ matrix
B1 = working copy of B
U = $r * k$
V = $c * k$
S = k-element vector
v = c-element vector
u = r-element vector
2. Compute $\mathbf{B} = \mathbf{A^T A}$
For each i, j:
$B_(ij)$ = sum over **p** of ($\mathbf{A}_(\mathbf{pi}) * \mathbf{A}_(\mathbf{pj})$)
3. Copy **B** into **B1** (**B1** will be deflated during iteration)
4. For each component $t = 0$ to $k - 1$:
a) eigen-val = pow-iter(**B1**, c, **v**)
b) S[t] = sqrt(eigen-val)
c) Store **v** as column t of **V**
d) Deflate $\mathbf{B1} = \mathbf{B1} - eigen - val \cdot \mathbf{v} \cdot \mathbf{v^T}$
(Removes the influence of this eigenvector)
5. Compute **U** (left singular vectors)
For each t = 0 to $k - 1$:
v = column t of **V**
$\mathbf{u} = \mathbf{A} \cdot \mathbf{v}$
normalize(**u**)
Store **u** as column t of **U**
6. Reconstruct A-recon = $\mathbf{USV^T}$
For each pixel position (b, d):
A-recon[b, d] = sum over $i = 0 \rightarrow k - 1$ of
U[**bi**]**S**[**i**]**V**[**di**]
7. Free all allocated memory

5) **Python :**

a) Load grayscale image and convert to float64 matrix **A**.
b) Make **A** C-contiguous for ctypes.
c) Create an empty matrix $\mathbf{A_r}$ of same size.
d) Load shared library svd-compression.so.
e) Define C function argument types and restype = None.
f) Set k = min(k, rows, cols).
g) Call svd-reconstructed(A, rows, cols, k, $\mathbf{A_r}$).
h) Compute Frobenius error = sqrt(sum($(A - A_r)^2$)).
i) Print k and error value.
j) Compare original and reconstructed images

VI. Implementation

Let us take an example image for implementation
If we take an image "einstein.jpg" and set k=20 , we get the following results through the code

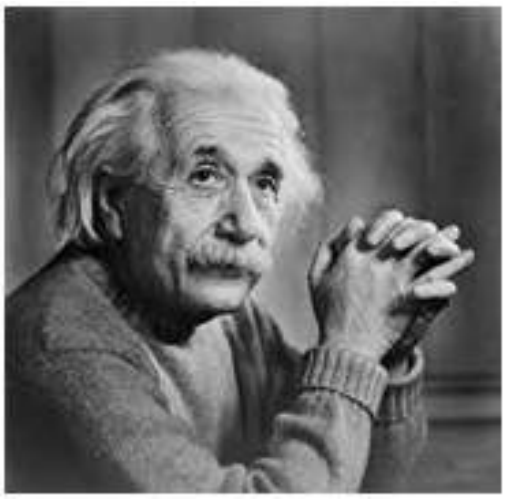

Fig. 1: Original

(a) k=5 (b) k=20 (c) k=50 (d) k=100

Similarly , we will get the results for remaining images with assigned k value of our choice .

VII. Error Analysis

The **Frobenius error** (or Frobenius norm of the difference) measures how close the reconstructed matrix $\mathbf{A_k}$ is to the original matrix $\mathbf{A}$

$$||\mathbf{E}||_F = ||\mathbf{A} - \mathbf{A_k}||_F$$

It keeps on changing for different images . As k increases → reconstruction improves → Frobenius error decreases.

k	Frobenius error
5	4713.338823
20	2126.439196
50	880.355637
100	164.818244

einstein.jpg

k	Frobenius error
5	20703.892759
20	10633.922961
50	6185.247394
100	3672.739859

globe.jpg

k	Frobenius error
5	11151.980266
20	3808.753741
50	1158.834821
100	511.562193

greyscale.png

TABLE I: Frobenius error for different images

VIII. Conclusion

1) This project successfully implemented a low-rank approximation of a grayscale image matrix using the **Truncated Singular Value Decomposition** (SVD), with the top k singular values and vectors computed entirely in C using the **Power Iteration** method with Deflation.
2) The choice of Power Iteration was justified by its straight forward implementation and relative ease of parallelization, though it necessitates working with the $\mathbf{A^T A}$ matrix, which can square the condition number and thus increase numerical error compared to direct SVD methods.
3) Despite this theoretical limitation, the hybrid C/Python implementation demonstrated effective and numerically stable extraction of the dominant eigenpairs required for compression.
4) The results, documented in the preceding sections, clearly illustrate the fundamental trade-off of image compression using SVD: as the rank k increases, the Frobenius norm error $||A - A_k||_F$ rapidly decreases, leading to higher image fidelity.